Algoritmi e Strutture Dati

Heap

Maria Rita Di Berardini, Emanuela Merelli[1]

[1]Dipartimento di Matematica e Informatica
Università di Camerino

Heap binari: definizione

Un **heap binario** è una albero binario quasi completo che soddisfa una delle due seguenti proprietà:

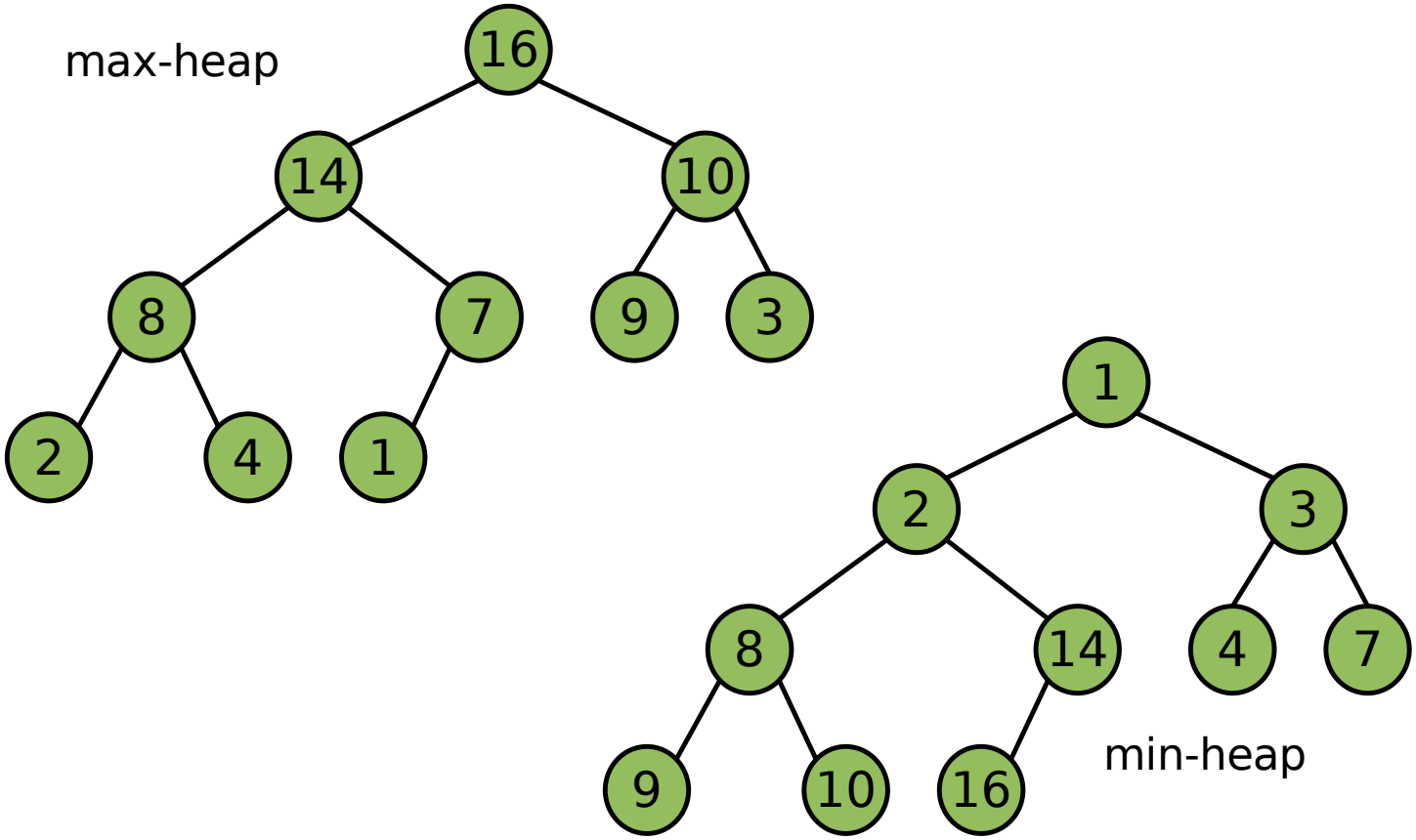
1. **proprietà del max-heap**: il valore associato a ciascun nodo u è maggiore o uguale al valore assiciato ai suoi figli – max-heap binario
2. **proprietà del min-heap**: il valore associato a ciascun nodo u è minore o uguale al valore assiciato ai suoi figli – min-heap binario

Un albero binario si dice quasi completo se:

* RIEMPITO DA SINISTRA A DESTRA

- tutti i livelli, tranne al più l'ultimo, sono completi
- nell'ultimo livello possono mancare delle foglie consecutive a partire però dall'ultima foglia a destra

Heap binari: esempi



Altezza di un heap binario

La "quasi" completezza garantisce che l'altezza h di un heap binario con è logaritmica nel numero di nodi, i.e

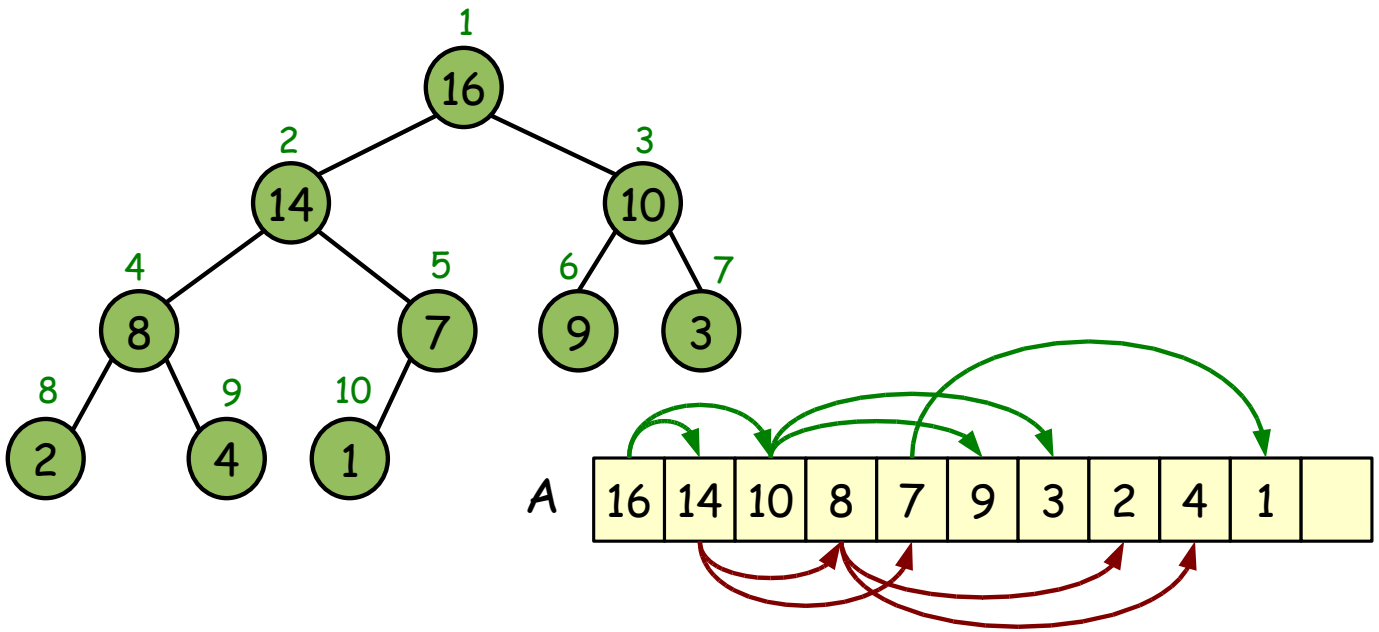
$$h = \Theta(\log_2 n)$$

Le operazioni sugli heap binari richiedono un tempo che è, al più, proporzionale alla altezza e, quindi, vengono eseguite in un tempo

$$O(\log_2 n)$$

Heap

Un heap binario può anche essere rappresentato come un'array



Heap

Un array che rappresenta un heap binario è un oggetto con due attributi:

- $length[A]$ è il numero di elementi (la dimensione) dell'array
- $heap\text{-}size[A]$ è il numero reale di elementi dell'heap memorizzati nell'array

La radice dell'albero è $A[1]$ (si trova in posizione 1)

Se i è l'indice di un dato nodo, gli indici del padre $parent[i]$, del figlio sinistro $left[i]$ e del figlio destro $right[i]$ sono $\lfloor i/2 \rfloor$, $2i$ e $2i+1$

Max-heap e min-heap

Proprietà del max-heap: in un max-heap ogni nodo i diverso dalla radice (i.e. $i \neq 1$) è tale che

$$A[parent[i]] \geq A[i]$$

Note:

1. il più grande elemento di un max-heap è memorizzato nella radice
2. un sottoalbero di un nodo u contiene nodi il cui valore non è mai **maggiore** del valore del nodo u

Max-heap e min-heap

Proprietà del min-heap: in un min-heap ogni nodo i diverso dalla radice ($i \neq 1$)è tale che

$$A[parent[i]] \leq A[i]$$

1. il più piccolo elemento di un min-heap viene memorizzato nella radice
2. un sottoalbero di un nodo u contiene nodi il cui valore non è mai **minore** del valore del nodo u

Code di Priorità (Priority Queues)

Un **coda di priorità** è una struttura dati che serve per mantenere un'insieme S di elementi ciascun con un valore detto **chiave** che rappresenta la sua priorità

Analogamente agli heap, esistono due tipi di code di priorità: code di max-priorità e code di min-priorità

Una coda di max-priorità supporta le seguenti operazioni:

1. **Insert**(S, x) inserisce x nell'insieme S
2. **Maximum**(S) restituisce l'elemento di S con chiave più grande
3. **Extract-Max**(S) elimina e restituisce l'elemento di S con chiave più grande
4. **Increase-Key**(S, x, k) aumenta il valore della chiave di x al nuovo valore k (qui assumiamo $k \geq$ dell'attuale valore della chiave di x)

Code di Priorità (Priority Queues)

Applicazioni di una coda di max-priorità: programmare i lavori su un computer condiviso (Job-Scheduling)

La coda di max-priorità tiene traccia dei lavori da svolgere e delle relative priorità

Quando un lavoro è ultimato o interrotto viene selezionato, mediante **Extract-Max**, il lavoro con priotità più alta tra quelli in attesa

Un nuovo lavoro puù essere aggiunto in qualsiasi momento mediante una **Insert**

Code di Priorità (Priority Queues)

In alternativa, una coda di min-priorità supporta le seguenti operazioni:

1. **Insert**(S, x) inserisce x nell'insieme S
2. **Minimum**(S) restituisce l'elemento di S con chiave minima
3. **Extract-Min**(S) elimina e restituisce l'elemento di S con chiave minima
4. **Decrease-Key**(S, x, k) decrementa il valore della chiave di x al nuovo valore k (qui assumiamo $k \leq$ dell'attuale valore della chiave di x)

Code di Priorità (Priority Queues)

Applicazioni di una coda di min-priorità: simulatore controllato di eventi (event-driven simulator)

Gli elementi della coda sono gli eventi da simulare; ad ognuno di essi è associata una chiave che rappresenta il tempo in cui l'evento si verifica

Gli eventi devono essere simulati secondo l'ordine dei loro tempi, questo perchè un evento può generare altri eventi

Ad ogni passaggio il programma seleziona l'evento da simulare mediante una **Etract-Min**

Ogni nuovo evento viene inserito in coda tramite una **Insert**

Vediamo come è possibile implementare una coda di max-priorità tramite un max-heap

Inserimento

Insert$(A, 15)$

	1	2	3	4	5	6	7	8	9	10	
A	16	14	10	8	7	9	3	2	4	1	

	1	2	3	4	5	6	7	8	9	10	i=11	
A	16	14	10	8	7	9	3	2	4	1	15	

A[parent[i]] < A[i]

	1	2	3	4	i=5	6	7	8	9	10	11	
A	16	14	10	8	15	9	3	2	4	1	7	

A[parent[i]] < A[i]

	1	i=2	3	4	5	6	7	8	9	10	11	
A	16	15	10	8	14	9	3	2	4	1	7	

A[parent[i]] ≥ A[i]

Max-Heap-Insert(A, x)

Max-Heap-Insert(A, x)

$heap\text{-}size[A] \leftarrow heap\text{-}size[A] + 1$

▷ aumentiamo la dimensione dell'heap

$i \leftarrow heap\text{-}size[A]$

$A[i] \leftarrow x$

while $i > 1$ and $A[parent[i]] < A[i]$

do scambia $A[i] \leftrightarrow A[parent[i]]$

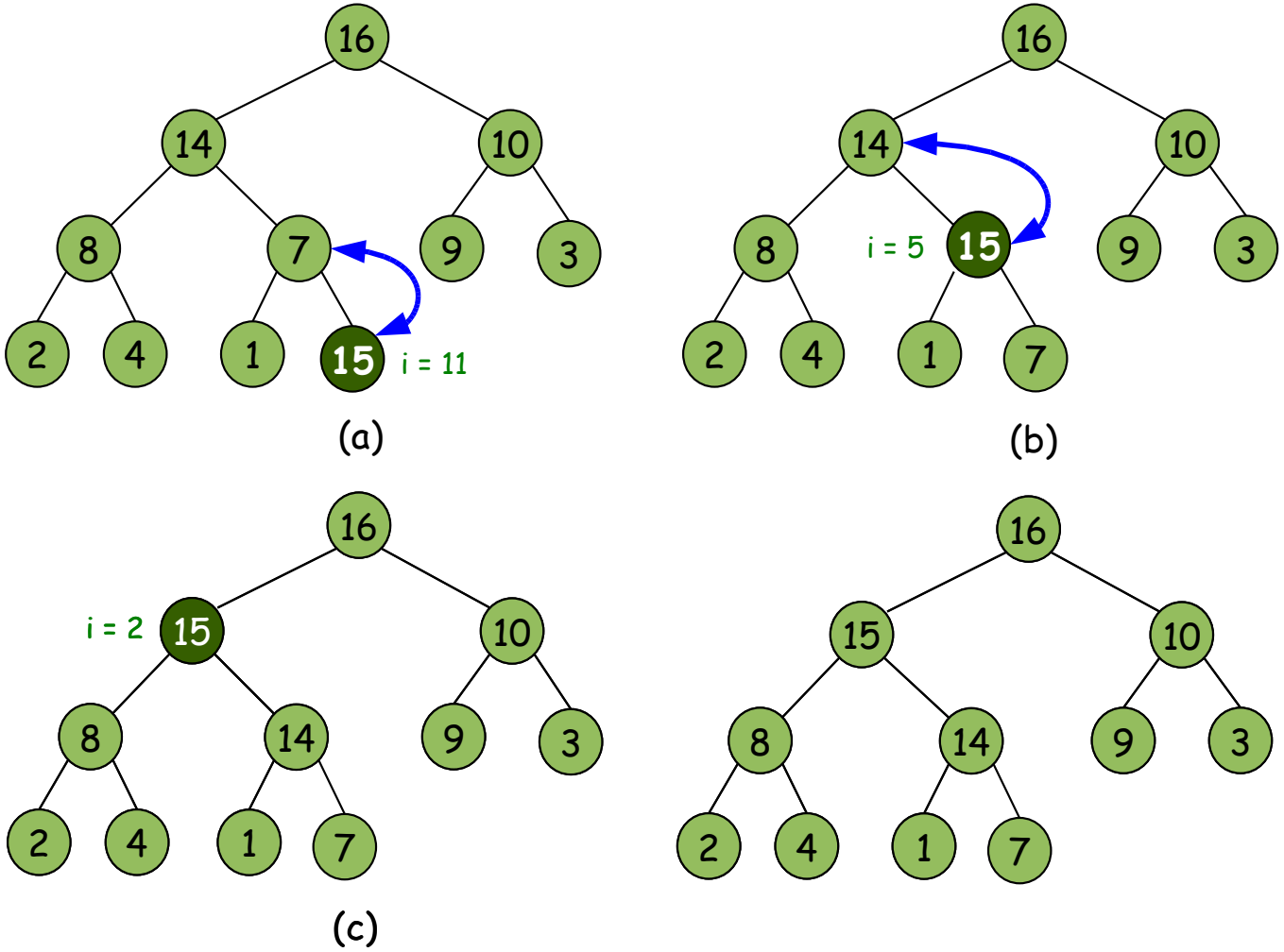
$i \leftarrow parent[i]$

Il numero di scambi è pari al più all'altezza dello heap, quindi

Max-Heap-Insert viene eseguita in un tempo

$$O(\log_2 n)$$

Max-Heap-Insert(A, x)

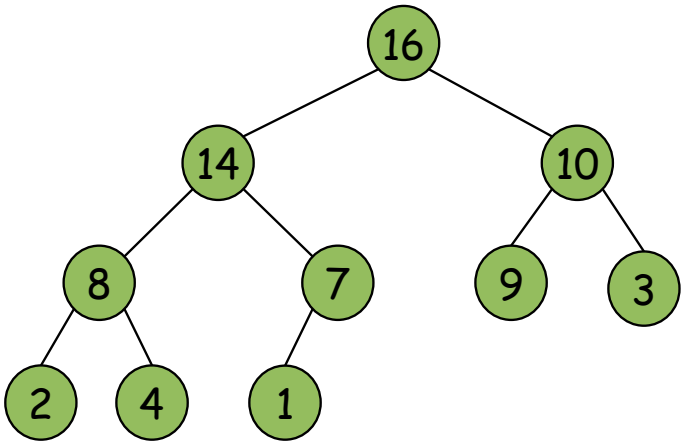


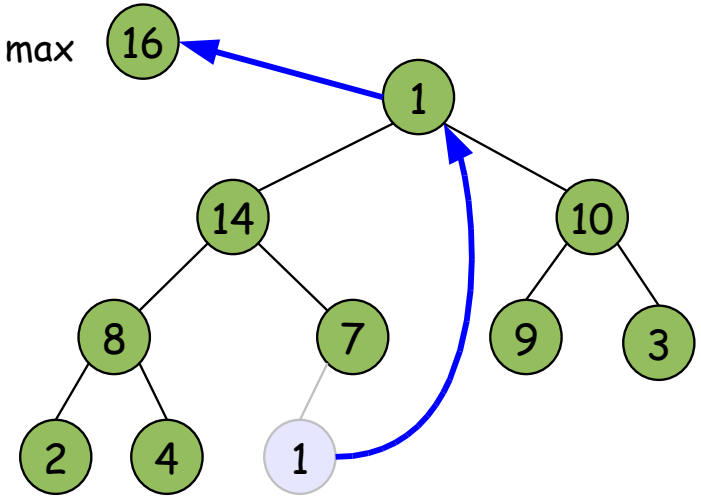
Restituzione del massimo

La seguente procedura implementa l'operazione **Maximum** in un tempo costante ($\Theta(1)$)

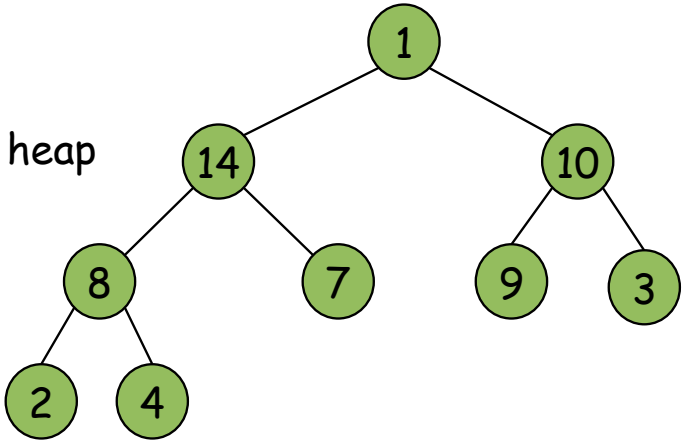
Heap-Maximum(A)
return $A[1]$

Estrazione del massimo





Non è un heap



Max-Heapify(*A*,1)

Max-Heapify(A, i)

È un importante subroutine per manipolare max-heap

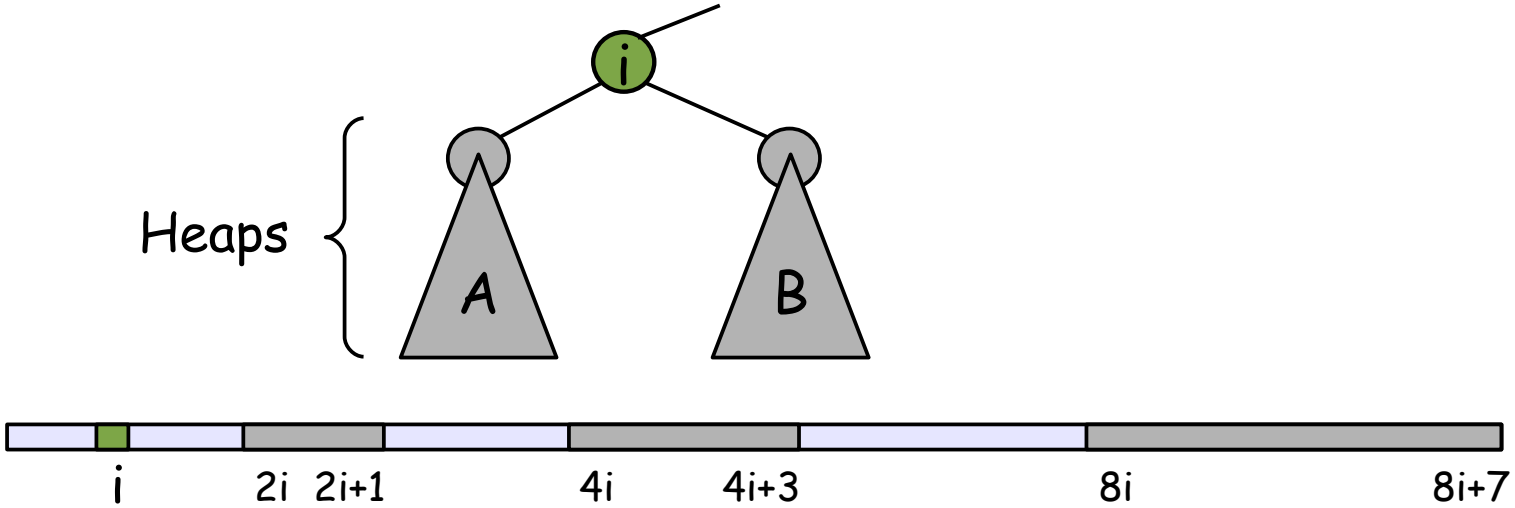
Input: un heap A e un indice i

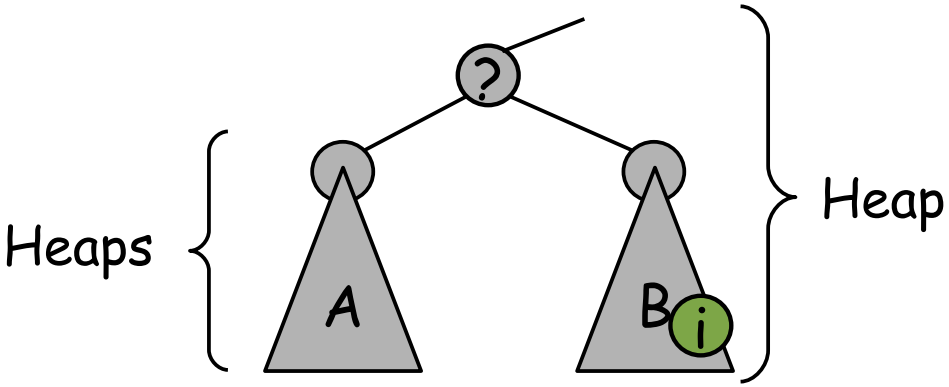
Max-Heapify(A, i) viene chiamata quando

- i sotto-alberi binari con radice $left[i]$ e $right[i]$ sono max-heap,
- ma $A[i]$ è più piccolo dei suoi figli, violando così la proprietà del max-heap

Idea: far scendere il valore di $A[i]$ nel max-heap in modo da ripristinare la proprietà desiderata

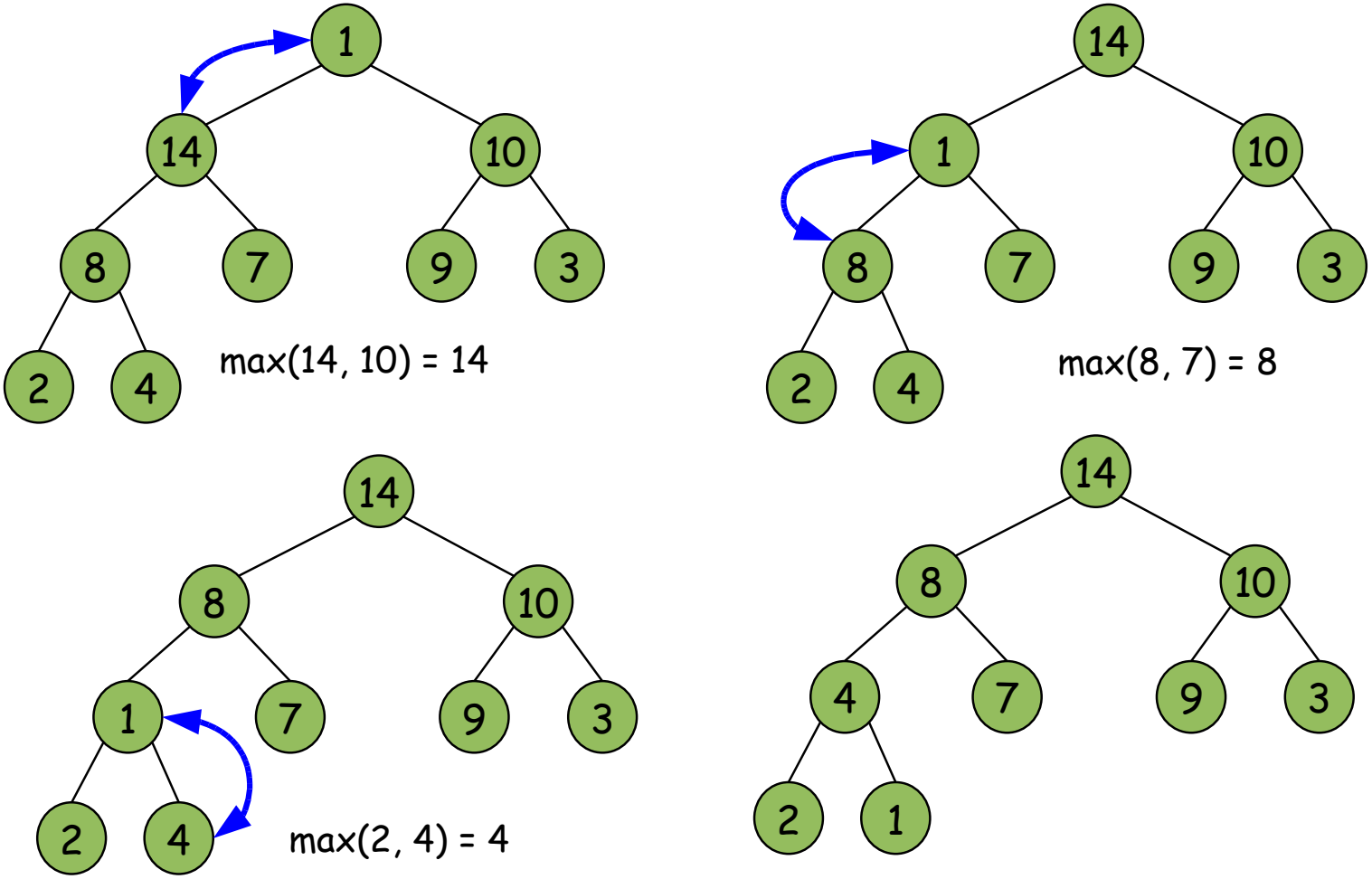
Max-Heapify(A, i)





Facciamo scendere il nodo i nell'albero fino a trovare la sua posizione

Max-Heapify(A, i)



Max-Heapify(A, i)

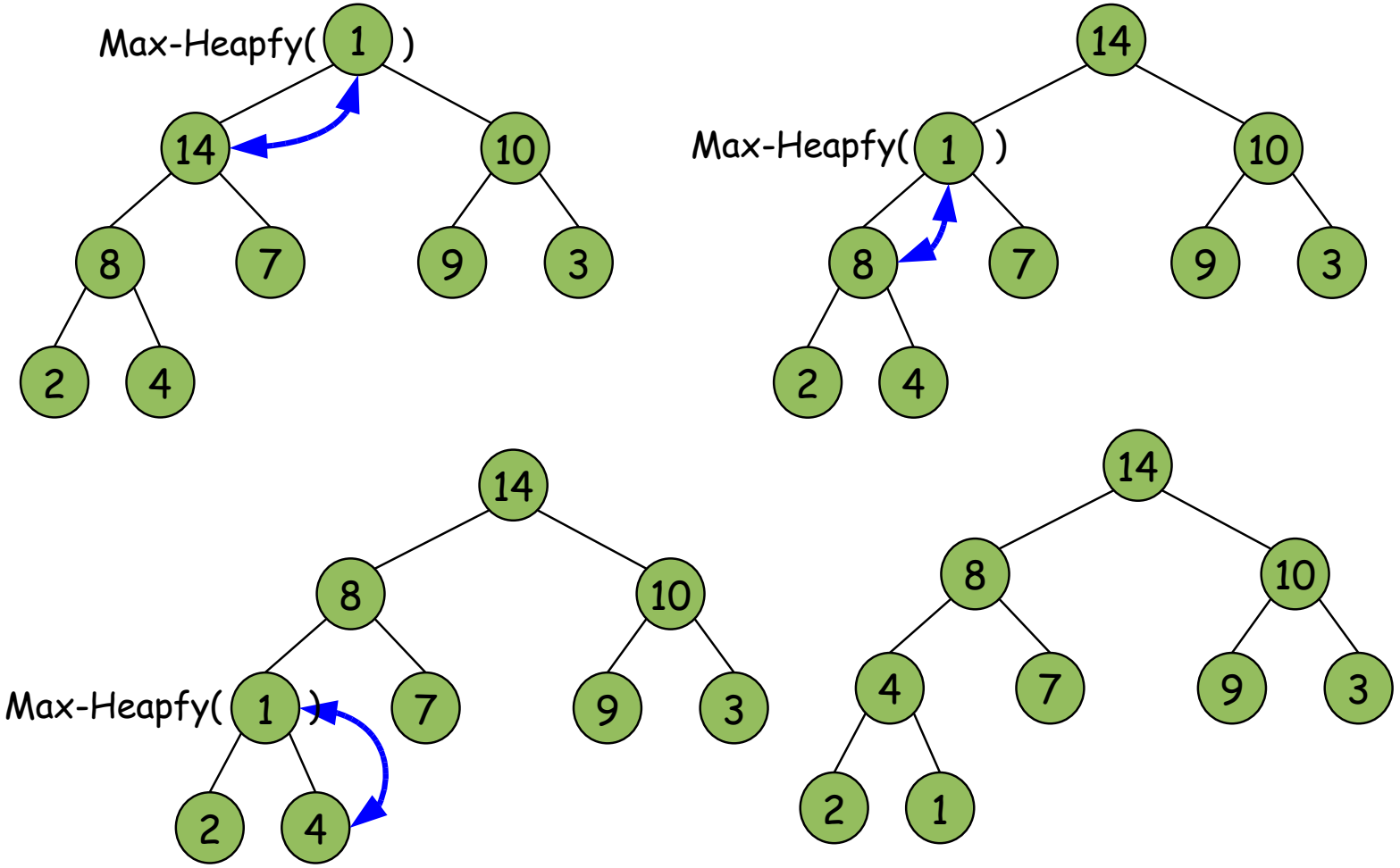
Max-Heapify(A, i)
  $l \leftarrow left[i]$
  $r \leftarrow right[i]$

  if $l \leq heap\text{-}size[A]$ and $A[l] > A[i]$
    then $max \leftarrow l$
    else $max \leftarrow i$

  if $r \leq heap\text{-}size[A]$ and $A[r] > A[max]$
    then $max \leftarrow r$

  if $max \neq i$
    then scambia $A[i] \leftrightarrow A[max]$
      Max-Heapify(A, max)

Max-Heapify(A, i)



Costo computazionale di Max-Heapify(A, i)

Il costo di **Max-Heapify** in un (sotto)albero di dimensione n è pari:

- al tempo costante $\Theta(1)$ necessario per stabilire il massimo fra gli elementi $A[i]$, $A[left(i)]$ e $A[right(i)]$
- più il tempo per eseguire Max-Heapify in un sottoalbero con radice uno dei figli di i

La dimensione di ciascun sottoalbero non supera $2n/3$

Il tempo di esecuzione di Max-Heapify può essere espresso dalla seguente ricorrenza

$$T(n) < T(2n/3) + \Theta(1)$$

Per il teorema master, la soluzione di questa ricorrenza è $O(\log_2 n)$

Esercizio

Risolvere la seguente ricorrenza usando il teorema dell'esperto

$$T'(n) = \begin{cases} 1 + T'(2n/3) & \text{se } n > 1 \\ 1 & \text{se } n = 1 \end{cases}$$

In questo caso $a = 1$ e $b = 3/2$ ($n/b = 2/3n$ implica $b = 3/2$).
Inoltre, $\log_b a = \log_b 1 = 0$ e $f(n) = 1 = \Theta(n^{\log_b a})$

Per il caso 2 del teorema dell'esperto abbiamo che

$$T'(n) = \Theta(n^{\log_b a} \log_2 n) = \Theta(\log_2 n)$$

Infine, poichè $T(n) \leq T'(n)$ implica $T(n) = \Theta(\log_2 n)$

Estrazione del massimo

Heap-Extract-Max(A)
  if $heap\text{-}size[A] < 1$
    then error "underflow dell'heap"
  $max \leftarrow A[1]$
  $A[1] \leftarrow A[heap\text{-}size[A]]$
  $heap\text{-}size[A] \leftarrow heap\text{-}size[A] - 1$
  Max-Heapify(A, 1)
  return max

Il costo computazionale è dato chiaramente dal costo della procedura Max-Heapify, ossia $O(\log_2 n)$

Incremento di una chiave

Increase-Key$(A, 9, 15)$

A	16	14	10	8	7	9	3	2	4	1

	1	2	3	4	5	6	7	8	i=9	10
A	16	14	10	8	7	9	3	2	15	1

A[parent[i]] < A[i]

	1	2	3	i=4	5	6	7	8	9	10
A	16	14	10	15	7	9	3	2	8	1

A[parent[i]] < A[i]

	1	i=2	3	4	5	6	7	8	9	10
A	16	15	10	14	7	9	3	2	8	1

A[parent[i]] ≥ A[i]

Heap-Increase-Key(A, i, key)

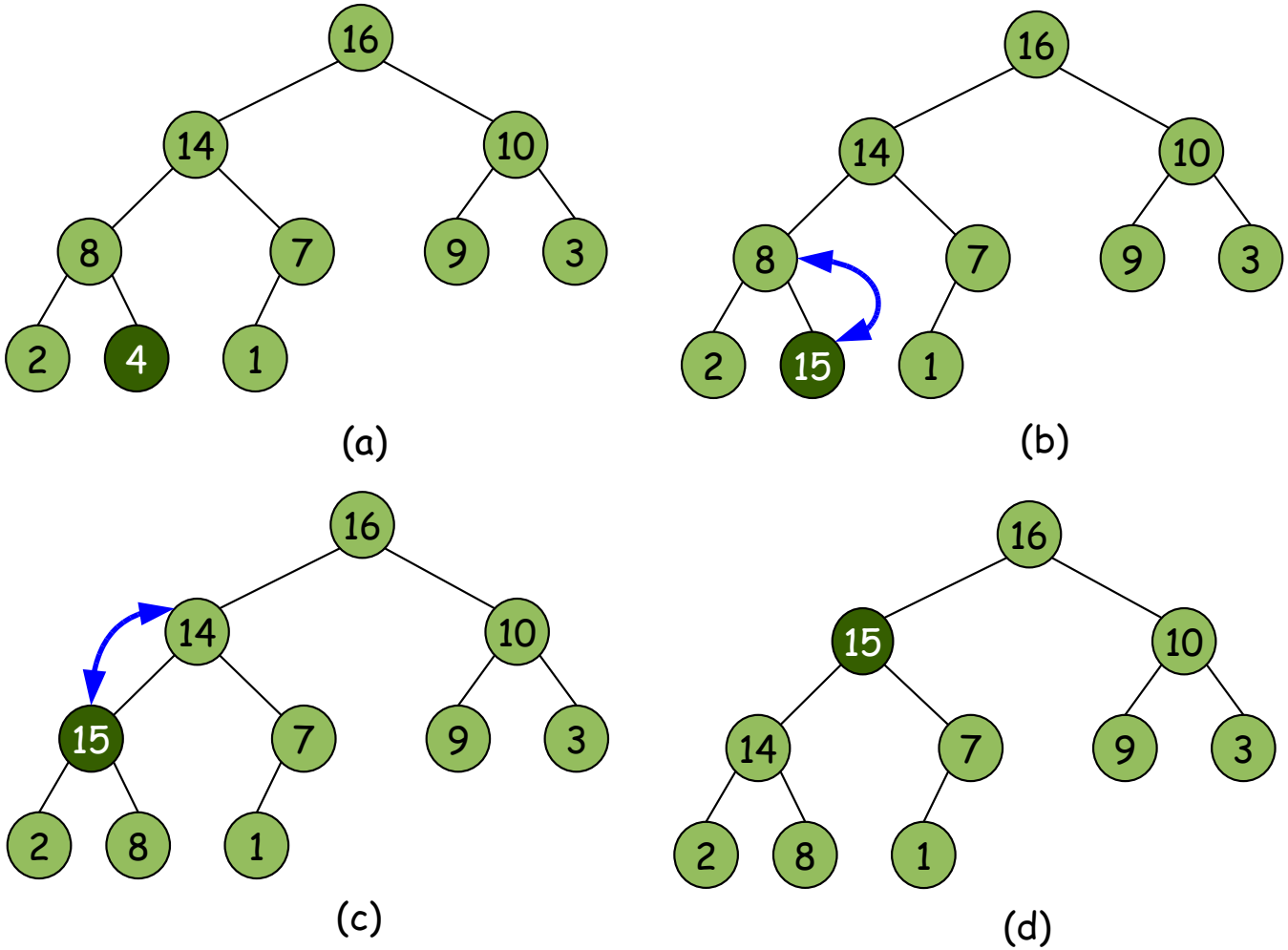
È molto simile alla **Max-Heap-Insert**

Heap-Increase-Key(A, i, key)
  if $key < A[i]$
    then error "la nuova chiave è più piccola di quella corrente"
  $A[i] \leftarrow key$
  while $i > 1$ or $A[parent[i]] < A[i]$
    do scambia $A[i] \leftrightarrow A[parent[i]]$
      $i \leftarrow parent[i]$

Costo computazionale:

$$O(\log_2 n)$$

Incremento di una chiave



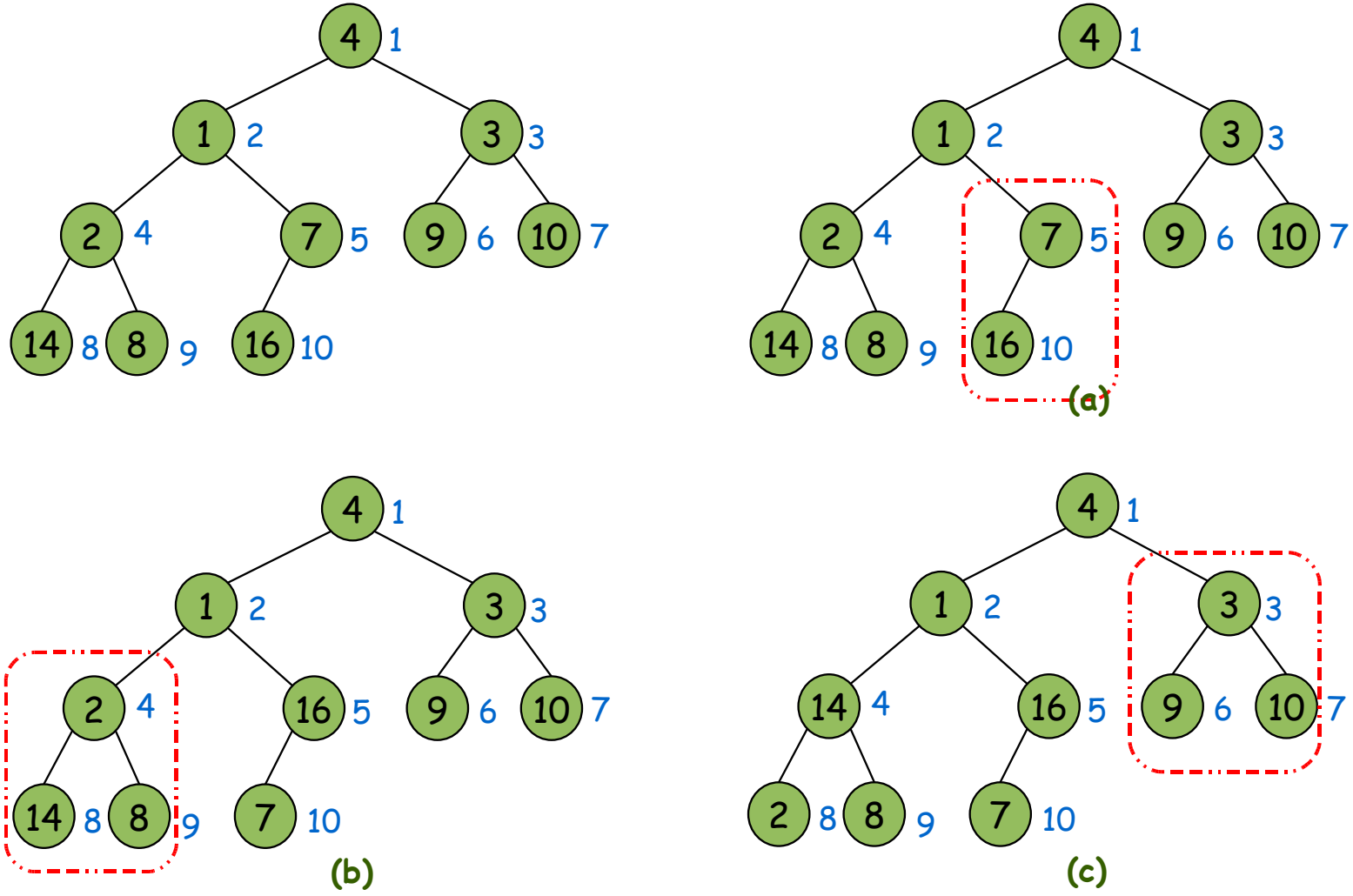
Costruzione di un heap

Possiamo usare la procedura **Max-Heapify** dal basso verso l'alto (in maniera bottom-up) per convertire un array $A[1 \dots n]$ (non "heapizzato") in un heap

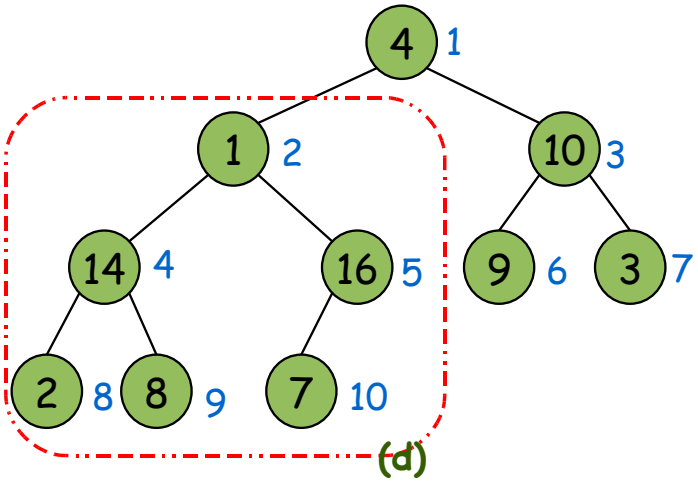
In un albero che rappresenta un heap di n elementi le foglie sono i nodi con indici $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$

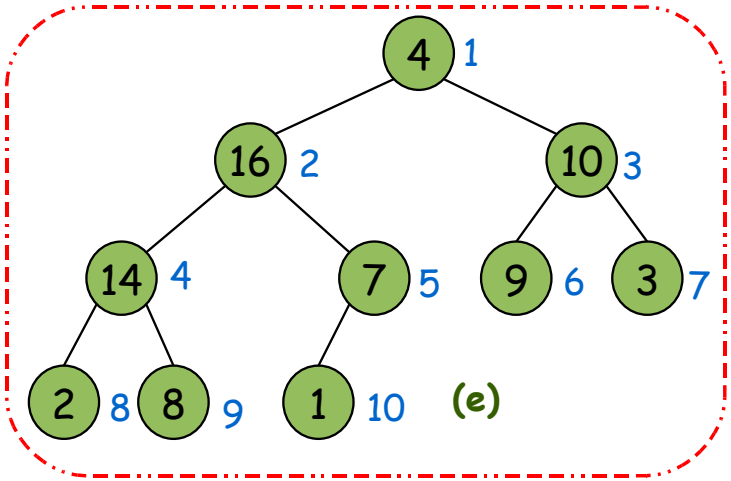
Ciascuno di essi è un heap di un elemento da cui cominciare

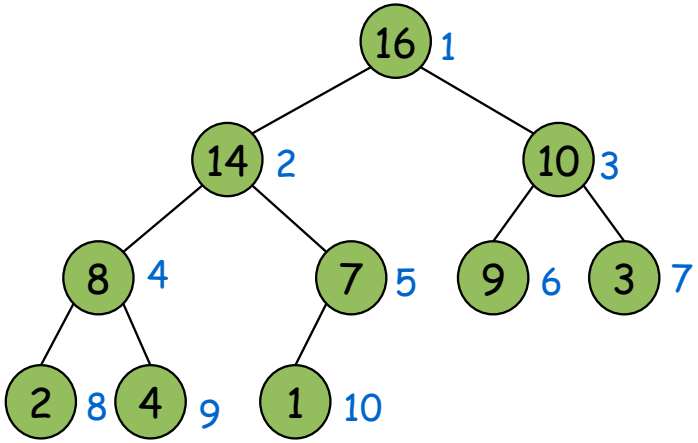
Costruzione di un heap



Costruzione di un heap







Build-Max-Heap(A)

Build-Max-Heap(A)
  $heap\text{-}size[A] \leftarrow length[A]$
  for $i \leftarrow \lfloor length[A]/2 \rfloor$ **downto** 1
    do Max-Heapify(A, i)

Il tempo di esecuzione di ogni chiamata di **Max-Heapify** è (al più) $O(\log_2 n)$. Eseguiamo $\lfloor length[A]/2 \rfloor = O(n)$ chiamate di **Max-Heapify**. Quindi, il tempo di esecuzione è $O(n \log_2 n)$

Build-Max-Heap(A)

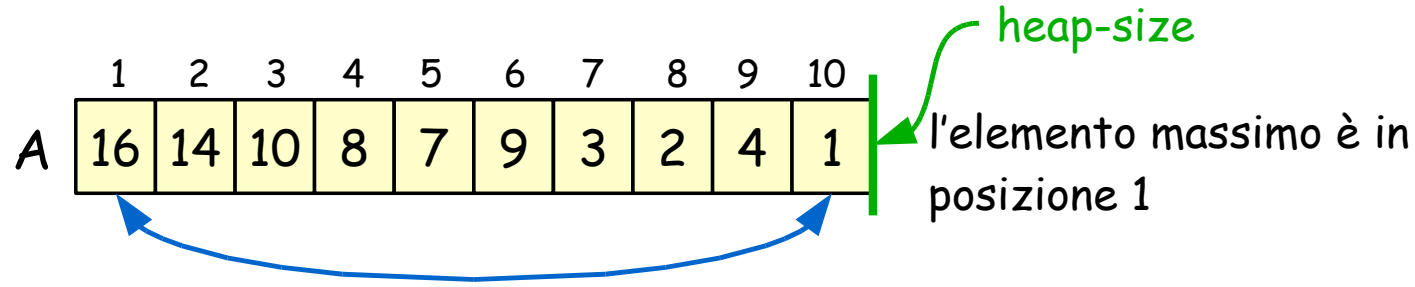
Il tempo di esecuzione di ogni chiamata di **Max-Heapify** è $O(n \log_2 n)$

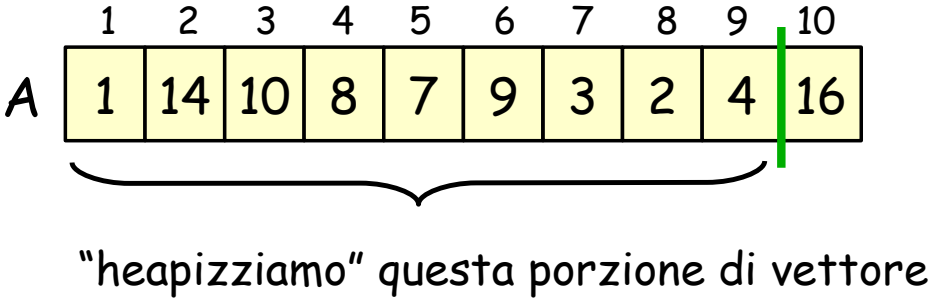
Questo limite superiore, sebbene corretto, non è asintoticamente stretto

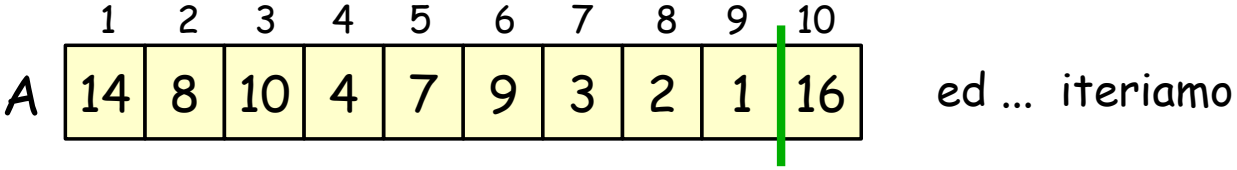
Possiamo fare di meglio osservando che il tempo di esecuzione di **Max-Heapify** varia al variare dell'altezza del nodo nell'heap e che le altezze della maggiorparte dei nodi sono piccole

Queste considerazioni portano a dimostrare che in il costo di ogni chiamata di **Max-Heapify** è $O(n)$

Ordinamento di un heap







Ordinamento di un heap

	1	2	3	4	5	6	7	8	9	10
A	14	8	10	4	7	9	3	2	1	16

il secondo elemento
più grade è in posizione 1

	1	2	3	4	5	6	7	8	9	10
A	1	8	10	4	7	9	3	2	14	16

"heapizziamo" questa porzione di vettore

	1	2	3	4	5	6	7	8	9	10
A	10	8	9	4	7	1	3	2	14	16

iterando n-1 volte
otteniamo un vettore
ordinato

Heapsort(A)

Heapsort(A)
Build-Max-Heap(A)
  for $i \leftarrow$ *length*[A] **downto** 2
    do scambia $A[1] \leftrightarrow A[i]$
      heap-size[A] $\leftrightarrow$ *heap-size*[A] $- 1$
      Max-Heapify($A, 1$)

La chiamata di **Build-Max-Heap** impiega $O(n)$ e ciascuna delle $n - 1$ di **Max-Heapify** impiega $O(\log_2 n)$

La procedura **Heapsort** impiega un tempo $O(n \log_2 n)$